A Study of Edge Orientation in Graph Neural Networks for Link Prediction

Nikita Scheglov
Advisor: Victor Kitov, PhD

Abstract

This study examines how retaining edge orientation during message passing affects directed link prediction with graph neural networks. We compare a feature-only multilayer perceptron, a symmetrized mean-aggregation encoder and an encoder with separate incoming and outgoing aggregation, using the same order-sensitive decoder architecture and a further symmetrized control with a matched parameter budget. The synthetic experiment uses 180 vertices with three latent roles and noisy role-related features, while varying directional structure over five levels and retaining the same undirected skeleton and features within each of five random seeds. Unordered vertex pairs are separated into training, validation and test groups, and both orientations of each training query are masked before message passing to prevent direct edge information from entering its prediction. Across 100 model runs, checkpoints are selected by validation average precision and evaluated using test ROC-AUC and average precision under a fixed one-to-one negative sampling ratio. At maximum directional strength, the directed encoder achieves ROC-AUC 0.723 compared with 0.575 for the capacity control, with a paired mean difference of 0.148 across seeds. The advantage is absent at weak directional structure, indicating that orientation-aware aggregation is useful under specific properties of this generating family rather than uniformly beneficial, and motivating subsequent evaluation on real directed networks.

Keywords: directed graphs, graph neural networks, link prediction, edge orientation.

1 Introduction

Directed graphs describe relations in which exchanging the source and target changes the meaning of an edge, as in citation and hyperlink networks. Link prediction on such graphs requires estimating the existence of an ordered edge rather than merely deciding whether two vertices are connected. The objective of this study is to assess how retaining edge orientation during neighborhood aggregation affects this prediction task.

GraphSAGE provides a simple inductive aggregation mechanism [1], while Dir-GNN explicitly separates incoming and outgoing neighborhoods [2]. Spectral approaches such as MagNet encode direction through complex-valued operators [3]. For link prediction, gravity-inspired and directed graph autoencoders additionally introduce asymmetric scoring functions [4, 5]. These studies establish relevant encoder and decoder designs, but the contribution of each component must be distinguished in a comparison.

An asymmetric decoder can assign different scores to reversed pairs even when its input embeddings were obtained from a symmetrized graph. Conversely, a symmetric inner-product decoder cannot express this distinction solely because its encoder preserves orientation [6, 7]. Moreover, reported prediction quality depends on data splits, negative sampling and baseline tuning [8]. The present

study therefore focuses on an encoder comparison with a common order-sensitive decoder and a shared evaluation protocol.

The working hypothesis is that separate incoming and outgoing aggregation improves link prediction when direction carries information about vertex roles. The study compares a feature-only model, a symmetrized mean-aggregation encoder and a directed encoder, and examine how their quality changes as directional structure varies. The contribution sought is a controlled characterization of this dependence; the study does not propose a new message-passing architecture.

The first experiment uses synthetic static graphs with fixed size and expected density and adjustable directional structure. The compared models receive identical vertex features, data splits and sampled negative pairs for each random seed. ROC-AUC and average precision measure prediction quality, with validation used for model selection and the test set reserved for final evaluation. This experiment is intended to identify the conditions under which preserving orientation is useful before extending the comparison to an open real-world graph.

Table 1: Reference approaches and their role in the comparison.

Approach	Strength	Limitation for this comparison
GraphSAGE [1]	Simple inductive neighborhood aggregation	Symmetrization discards the orientation of messages.
Dir-GNN [2]	Separates incoming and outgoing neighborhoods	Its classification results do not establish link-prediction gains.
Gravity-inspired autoencoder [4]	Scores ordered pairs asymmetrically	Changing the decoder does not isolate the effect of the encoder.

2 Related work

Message passing. Message-passing networks separate message construction, neighborhood aggregation and readout [9]. GCN, GraphSAGE and GAT provide standard aggregation mechanisms [1, 10, 11], while GIN and GATv2 clarify the expressive limitations of aggregation and attention [12, 13]. Mean aggregation offers a simple reference for comparing a shared undirected neighborhood with separate incoming and outgoing neighborhoods.

Directed representations. Dir-GNN separates incoming and outgoing messages and studies their benefit for node classification [2]. DGCN and DiGCN incorporate directed proximity and diffusion [14, 15], whereas DEDGAT and ADPA distinguish directional roles and patterns [16, 17]. These classification results motivate a directed encoder but do not establish an improvement in link prediction. MagNet encodes orientation through a magnetic Laplacian [3]; MSGNN and SigMaNet extend spectral modeling to signed directed graphs [18, 19]. DUPLEX combines directional attention and complex embeddings [20]. DiGCL and S2-DiGCL introduce contrastive learning for directed representations [21, 22]. Directed positional encodings provide another route to retaining directional structure [23, 24].

Asymmetric decoding. The standard inner-product decoder of a variational graph autoencoder assigns identical scores to reversed pairs [6]. Gravity-inspired autoencoders introduce an asymmetric decoder [4], while directed graph autoencoders learn source and target representations [5]. LINE and node2vec provide alternative network-embedding references [25, 26]. For a fixed relation, DistMult uses a symmetric score; ComplEx can represent asymmetric relations [7, 27]. Consequently, directed message passing and asymmetric edge scoring are distinct design choices.

Pairwise structure. SEAL predicts links from enclosing subgraphs, whereas NBFNet learns query-dependent path representations [28, 29]. Neo-GNNs and Neural Common Neighbor exploit neighborhood overlap [30, 31]; ELPH and BUDDY approximate link-specific structure through sketching [32]. TNCN extends common-neighbor representations to temporal graphs [33]. These approaches motivate structural controls, but introduce additional mechanisms beyond changing edge orientation in the encoder.

Data and evaluation. H2GCN and LINKX study learning beyond homophily [34, 35]; heterophily and edge directionality should be treated as separate properties. Platonov et al. identify dataset weaknesses and leakage in heterophily evaluation [36]. OGB supplies task-specific splits and evaluation protocols [37], and HeaRT highlights the sensitivity of link prediction to tuning, splits and negative sampling [8]. Dynamic link-prediction protocols and TGB benchmarks further demonstrate the importance of matching evaluation to the prediction task [38–40]. The initial study therefore considers static directed edge recovery with shared splits and negative examples; temporal and multi-relational prediction are separate extensions.

Scope of the comparison. The study focuses on a controlled comparison of directional neighborhood aggregation. A common order-sensitive decoder is used with feature-only, symmetrized and directed encoders. The synthetic experiment varies directional structure while keeping graph size and expected density fixed, with prediction quality summarized across random seeds. This design separates the contribution of the encoder from decoder asymmetry and evaluation choices.

3 Directed edge recovery

Given a partially observed directed graph and vertex features, the task is to predict whether an ordered pair of distinct vertices forms an edge. The initial setting is transductive: all vertices are available during training, but validation and test edges are withheld. Graphs are static, unweighted and contain a single edge type; self-loops are excluded from the prediction task.

3.1 Data and notation

Let $V = \{1, \dots, n\}$ and let $\mathcal{U} = \{(u, v) \in V^2 : u \neq v\}$ be the candidate ordered pairs. The full synthetic graph is $G = (V, E)$, where $E \subseteq \mathcal{U}$, and $X \in \mathbb{R}^{n \times d}$ contains vertex features. The target is $y_{uv} = \mathbf{1}\{(u, v) \in E\}$. In particular, y_{uv} and y_{vu} need not coincide; a reversed edge is not automatically a negative example. For synthetic graphs, absent pairs have known labels under the specified generator.

Table 2: Notation used in the problem statement.

Symbol	Meaning
$G = (V, E)$	Full synthetic directed graph
$X,\ x_v$	Feature matrix and feature vector of vertex v
$\rho \in [0, 1]$	Strength of directional structure in the graph generator
$E_{\mathrm{tr}}, E_{\mathrm{va}}, E_{\mathrm{te}}$	Training, validation and test positive edges
$N_{\mathrm{tr}}, N_{\mathrm{va}}, N_{\mathrm{te}}$	Corresponding sampled negative ordered pairs
A_{tr}	Adjacency matrix of the training graph
$H,\ h_v$	Embedding matrix and embedding of vertex v
$\theta = (\phi, \psi)$	Encoder and decoder parameters

Synthetic instances are sampled from a family P_ρ that varies directional structure while keeping n and the undirected skeleton fixed across ρ. The generating distribution is specified in Section 4. The initial comparison assumes that these roles influence connectivity and that vertex features provide noisy information about them. For each ρ, graph instances, splits and initialization seeds are recorded. Conclusions from this setting are restricted to the specified generating family.

3.2 Training graph and labeled pairs

Unordered vertex pairs $\{u, v\}$ are randomly partitioned into training, validation and test groups with proportions 0.8, 0.1 and 0.1. Both orientations of a pair belong to the same group. Positive sets E_s contain the edges in group $s \in \{\mathrm{tr}, \mathrm{va}, \mathrm{te}\}$. Negative sets N_s are sampled uniformly without replacement from absent ordered pairs in that group, with $|N_s| = |E_s|$. Thus positive and negative examples are disjoint across splits and reciprocal positive edges cannot occur in different splits. Each labeled sample is

$$\mathcal{D}_s = \{(u, v, 1) : (u, v) \in E_s\} \cup \{(u, v, 0) : (u, v) \in N_s\}. \tag{1}$$

The message-passing adjacency satisfies $(A_{\mathrm{tr}})_{uv} = \mathbf{1}\{(u,v) \in E_{\mathrm{tr}}\}$. Validation and test labels are never supplied to the encoder, including through symmetrization or feature construction. For each training batch, both orientations of every query pair are removed from the message-passing graph before computing the loss, including reverse edges of negative queries. At validation and test time the encoder uses the fixed training graph only.

3.3 Model class

For $m \in \mathcal{M} = \{\mathrm{MLP}, \mathrm{Sym}, \mathrm{Dir}\}$ and an adjacency matrix A, the encoder produces $H = f_\phi^{(m)}(X, A) \in \mathbb{R}^{n \times h}$. The neighborhoods below refer to A_{tr}; for a masked adjacency they are formed from the remaining edges. The feature-only model processes each x_v independently. The symmetrized encoder aggregates over $\mathcal{N}_{\mathrm{sym}}(v) = \{u : (u,v) \in E_{\mathrm{tr}} \text{ or } (v,u) \in E_{\mathrm{tr}}\}$. The directed encoder aggregates incoming and outgoing neighborhoods separately:

$$\begin{aligned} \mathcal{N}_{\mathrm{in}}(v) &= \{u : (u,v) \in E_{\mathrm{tr}}\}, \\ \mathcal{N}_{\mathrm{out}}(v) &= \{u : (v,u) \in E_{\mathrm{tr}}\}. \end{aligned} \tag{2}$$

Mean aggregation over an empty neighborhood is defined as the zero vector. Both graph encoders retain a separate self-feature transformation, use the same number of layers and are derived from the GraphSAGE and Dir-GNN designs [1, 2]. Their widths and parameter counts are reported, with an additional parameter-budget control to assess the effect of model capacity.

All encoders use the same decoder architecture and input ordering:

$$p_\theta^{(m)}(u, v; A) = \sigma(g_\psi([h_u \| h_v])), \qquad \sigma(z) = (1 + e^{-z})^{-1}. \tag{3}$$

Here $[h_u \| h_v]$ denotes concatenation and g_ψ is a multilayer perceptron with a scalar output. Its parameters are learned separately for each model; the architecture is shared, not the fitted weights. The decoder permits $p_\theta^{(m)}(u, v; A) \neq p_\theta^{(m)}(v, u; A)$, without requiring this inequality for every pair.

3.4 Quality criteria and model selection

The internal criterion is mean binary cross-entropy with L_2 regularization, where $\lambda > 0$ is fixed within a comparison. The encoder parameters and decoder parameters are optimized jointly. Let $\mathfrak{B}$ be a recorded partition of $\mathcal{D}_{\mathrm{tr}}$ into batches. For each batch B, let $A_{\mathrm{tr}}^{(-B)}$ be the training adjacency with both orientations of every query pair removed, regardless of its label. The batch-specific probability and binary cross-entropy are

$$\begin{aligned} q_{\theta,B}^{(m)}(u,v) &= p_\theta^{(m)}(u, v; A_{\mathrm{tr}}^{(-B)}), \\ \ell(y,p) &= -y \log p - (1-y)\log(1-p). \end{aligned} \tag{4}$$

The external criteria are ROC-AUC and average precision (AP), computed on the same labeled pairs for all models. AP is the sum of precision values at successive score thresholds weighted by the corresponding increments in recall. ROC-AUC is the probability that a positive example is ranked above a negative one, counting ties as one half. The negative-to-positive ratio is fixed at $1:1$; AP values therefore describe this sampled evaluation distribution. Hyperparameters are fixed in the initial experiment; checkpoints are selected by validation AP under a common maximum epoch count and early-stopping rule. Test labels are used only for final evaluation.

For K recorded random seeds, the comparison reports the mean and sample standard deviation of each test metric. The directional effect is summarized by paired differences

$$\Delta_M(\rho) = \frac{1}{K} \sum_{k=1}^{K} \left(M_{\mathrm{Dir}}^{(k)}(\rho) - M_{\mathrm{Sym}}^{(k)}(\rho) \right), \qquad M \in \{\text{ROC-AUC}, \text{AP}\}, \tag{5}$$

using the same graph instance, features, split and negative pairs within each seed. A positive difference indicates improved performance under that configuration; an advantage is not assumed for every ρ.

For a model m and a fixed hyperparameter configuration, the training problem is

$$\boxed{\theta_m^* \in \operatorname*{arg\,min}_{\theta \in \Theta_m} \left\{ \frac{1}{|\mathcal{D}_{\mathrm{tr}}|} \sum_{B \in \mathfrak{B}} \sum_{(u,v,y) \in B} \ell\Big(y, q_{\theta,B}^{(m)}(u,v)\Big) + \lambda \|\theta\|_2^2 \right\}.} \tag{6}$$

4 Baseline experiment

Objective. The experiment tests whether directional neighborhood aggregation improves recovery of missing ordered edges when orientation reflects latent vertex roles. The skeleton, features and evaluation protocol are controlled to distinguish this effect from changes in graph size or model capacity.

Generating distribution. There are $n = 180$ vertices with three equally sized roles $c_v \in \{0, 1, 2\}$ assigned by a random permutation. For every unordered pair $\{u, v\}$, a skeleton edge is independently sampled with probability $q = 0.15$. Each sampled edge receives exactly one orientation, with

$$\Pr(u \to v \mid \{u, v\} \text{ sampled}, c_u, c_v) = \frac{1 + \rho C_{c_u c_v}}{2}, \qquad C = \begin{pmatrix} 0 & 1 & -1 \\ -1 & 0 & 1 \\ 1 & -1 & 0 \end{pmatrix}. \tag{7}$$

At $\rho = 0$ orientation is random; at $\rho = 1$ edges between different roles follow the cycle $0 \to 1 \to 2 \to 0$. Within-role edges retain random orientation. The same skeleton and orientation uniforms are reused across ρ within each seed, so the total edge count is unchanged. The expected number of edges is $qn(n-1)/2 = 2416.5$. Features are $x_v = e_{c_v} + \varepsilon_v \in \mathbb{R}^6$, where e_c is the coordinate vector for role c among the first three coordinates and $\varepsilon_v \sim \mathcal{N}(0, I_6)$ independently. Role indices are not supplied to the encoders. Realized graphs contain 2324–2474 directed edges.

Models and settings. The experiment uses $\rho \in \{0, 0.25, 0.5, 0.75, 1\}$ and seeds $0, \ldots, 4$. All encoders have two ReLU layers and produce 32-dimensional vertex embeddings. Their common decoder is a $64 \to 32 \to 1$ MLP applied to ordered concatenations, with a ReLU hidden activation. MLP, Sym and Dir use hidden width 32. SymWide uses hidden width 48, matching Dir's total parameter count to within 1% while retaining the same output dimension and decoder architecture. Sym and Dir implement mean aggregation with separate self-feature transformations following GraphSAGE and Dir-GNN [1, 2].

Adam uses learning rate 0.01, $\lambda = 10^{-4}$ and batches of at most 512 labeled pairs. Training is limited to 120 epochs and stops after 25 epochs without an improvement in validation AP. The best validation-AP checkpoint is selected; no hyperparameter search is performed. The dyad split and query masking follow Section 3. Each model uses identical samples within each graph instance. The implementation uses deterministic CPU operations with one thread.

Results. Table 3 reports the maximum-direction setting; Figure 1 includes every tested ρ. At $\rho = 1$, Dir improves over the capacity control by 0.148 ± 0.020 in ROC-AUC and 0.095 ± 0.009 in AP (paired means and sample standard deviations). At weak directional structure, the directed encoder has no consistent advantage. The main separation occurs at $\rho = 1$, supporting the hypothesis for this generating family rather than a uniform benefit from direction-aware aggregation. The results do not establish superiority on real-world graphs.

Table 3: Test quality at $\rho = 1$: mean and sample standard deviation over five seeds.

Model	Parameters	ROC-AUC	AP	Training BCE
Feature-only MLP	3393	0.594 ± 0.041	0.575 ± 0.037	0.476
Symmetrized	4609	0.529 ± 0.060	0.527 ± 0.042	0.646
Directed	5825	0.723 ± 0.030	0.655 ± 0.042	0.483
Symmetrized (wide)	5841	0.575 ± 0.050	0.559 ± 0.035	0.523

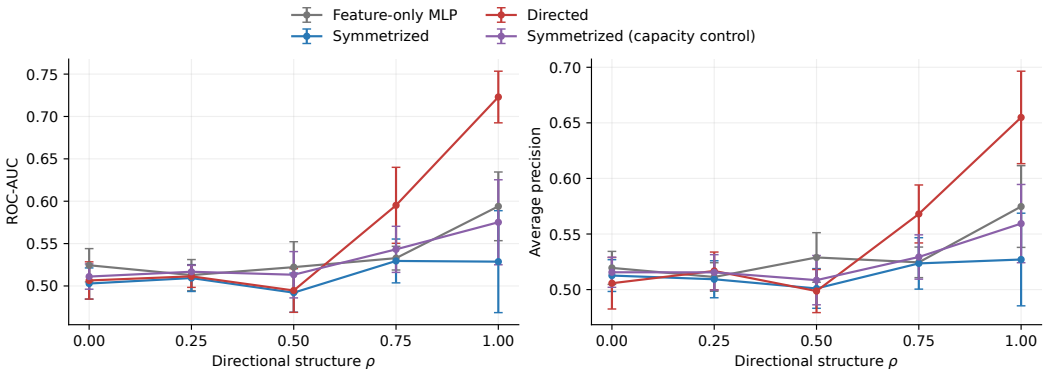


Figure 1: Test quality versus directional structure. Error bars show sample standard deviations over five seeds. All models use the same ordered-pair decoder architecture.

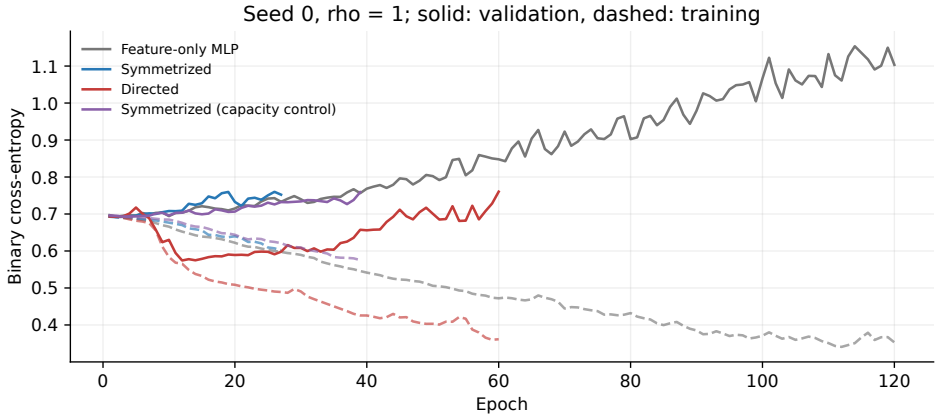


Figure 2: Loss curves for seed 0 at $\rho = 1$. Solid lines show validation BCE; dashed lines show mean batch training BCE during each epoch. Early stopping is based on validation AP, not BCE.

Reproduction. The command `python code/main.py` regenerates the data, numerical results and figures. Configuration is stored in `code/configs/baseline.json`; the plotting module reads saved numerical results independently of training. Per-run metrics, losses, epochs, parameter counts and timings are retained in CSV files, with software versions and source hashes in `results/metadata.json`. Total training and evaluation time for the 100 runs was 71.9 seconds on the recorded CPU environment; file writing and plotting are excluded. The hypothesis sketch is retained separately as `paper/figures/expected_curve.pdf` and contains no experimental measurements.

References

[1] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs, 2017. URL https://arxiv.org/abs/1706.02216.

[2] Emanuele Rossi, Bertrand Charpentier, Francesco Di Giovanni, Fabrizio Frasca, Stephan Günnemann, and Michael Bronstein. Edge Directionality Improves Learning on Heterophilic Graphs. In *Proceedings of the Second Learning on Graphs Conference*, volume 231 of *Proceedings of Machine Learning Research*, pages 25:1–25:27, 2024. URL https://proceedings.mlr.press/v231/rossi24a.html.

[3] Xitong Zhang, Yixuan He, Nathan Brugnone, Michael Perlmutter, and Matthew Hirn. MagNet: A Neural Network for Directed Graphs, 2021. URL https://arxiv.org/abs/2102.11391.

[4] Guillaume Salha, Stratis Limnios, Romain Hennequin, Viet Anh Tran, and Michalis Vazirgiannis. Gravity-Inspired Graph Autoencoders for Directed Link Prediction, 2019. URL https://arxiv.org/abs/1905.09570.

[5] Georgios Kollias, Vasileios Kalantzis, Tsuyoshi Ide, Aurélie Lozano, and Naoki Abe. Directed Graph Auto-Encoders. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 7211–7219, 2022. doi: 10.1609/aaai.v36i7.20682. URL https://ojs.aaai.org/index.php/AAAI/article/view/20682.

[6] Thomas N. Kipf and Max Welling. Variational Graph Auto-Encoders, 2016. URL https://arxiv.org/abs/1611.07308.

[7] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex Embeddings for Simple Link Prediction, 2016. URL https://arxiv.org/abs/1606.06357.

[8] Juanhui Li, Harry Shomer, Haitao Mao, Shenglai Zeng, Yao Ma, Neil Shah, Jiliang Tang, and Dawei Yin. Evaluating Graph Neural Networks for Link Prediction: Current Pitfalls and New Benchmarking, 2023. URL https://arxiv.org/abs/2306.10453.

[9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural Message Passing for Quantum Chemistry, 2017. URL https://arxiv.org/abs/1704.01212.

[10] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks, 2016. URL https://arxiv.org/abs/1609.02907.

[11] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks, 2017. URL https://arxiv.org/abs/1710.10903.

[12] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How Powerful are Graph Neural Networks?, 2018. URL https://arxiv.org/abs/1810.00826.

[13] Shaked Brody, Uri Alon, and Eran Yahav. How Attentive are Graph Attention Networks?, 2021. URL https://arxiv.org/abs/2105.14491.

[14] Zekun Tong, Yuxuan Liang, Changsheng Sun, David S. Rosenblum, and Andrew Lim. Directed Graph Convolutional Network, 2020. URL https://arxiv.org/abs/2004.13970.

[15] Zekun Tong, Yuxuan Liang, Changsheng Sun, Xinke Li, David Rosenblum, and Andrew Lim. Digraph Inception Convolutional Networks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/cffb6e2288a630c2a787a64ccc67097c-Abstract.html.

[16] Jiafu Wu, Mufeng Yao, Dong Wu, Mingmin Chi, Baokun Wang, Ruofan Wu, Xin Fu, Changhua Meng, and Weiqiang Wang. DEDGAT: Dual Embedding of Directed Graph Attention Networks for Detecting Financial Risk, 2023. URL https://arxiv.org/abs/2303.03933.

[17] Henan Sun, Xunkai Li, Zhengyu Wu, Daohan Su, Rong-Hua Li, and Guoren Wang. Breaking the Entanglement of Homophily and Heterophily in Semi-supervised Node Classification, 2023. URL https://arxiv.org/abs/2312.04111.

[18] Yixuan He, Michael Perlmutter, Gesine Reinert, and Mihai Cucuringu. MSGNN: A Spectral Graph Neural Network Based on a Novel Magnetic Signed Laplacian, 2022. URL https://arxiv.org/abs/2209.00546.

[19] Stefano Fiorini, Stefano Coniglio, Michele Ciavotta, and Enza Messina. SigMaNet: One Laplacian to Rule Them All, 2022. URL https://arxiv.org/abs/2205.13459.

[20] Zhaoru Ke, Hang Yu, Jianguo Li, and Haipeng Zhang. DUPLEX: Dual GAT for Complex Embedding of Directed Graphs, 2024. URL https://arxiv.org/abs/2406.05391.

[21] Zekun Tong, Yuxuan Liang, Henghui Ding, Yongxing Dai, Xinke Li, and Changhu Wang. Directed Graph Contrastive Learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/a3048e47310d6efaa4b1eaf55227bc92-Abstract.html.

[22] Zhengyu Wu, Daohan Su, Yang Zhang, Xunkai Li, Rong-Hua Li, and Guoren Wang. Toward General Digraph Contrastive Learning: A Dual Spatial Perspective, 2025. URL https://arxiv.org/abs/2510.16311.

[23] Simon Geisler, Yujia Li, Daniel Mankowitz, Ali Taylan Cemgil, Stephan Günnemann, and Cosmin Paduraru. Transformers Meet Directed Graphs, 2023. URL https://arxiv.org/abs/2302.00049.

[24] Yinan Huang, Haoyu Wang, and Pan Li. What Are Good Positional Encodings for Directed Graphs?, 2024. URL https://arxiv.org/abs/2407.20912.

[25] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. LINE: Large-scale Information Network Embedding, 2015. URL https://arxiv.org/abs/1503.03578.

[26] Aditya Grover and Jure Leskovec. node2vec: Scalable Feature Learning for Networks, 2016. URL https://arxiv.org/abs/1607.00653.

[27] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding Entities and Relations for Learning and Inference in Knowledge Bases, 2014. URL https://arxiv.org/abs/1412.6575.

[28] Muhan Zhang and Yixin Chen. Link Prediction Based on Graph Neural Networks, 2018. URL https://arxiv.org/abs/1802.09691.

[29] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural Bellman-Ford Networks: A General Graph Neural Network Framework for Link Prediction, 2021. URL https://arxiv.org/abs/2106.06935.

[30] Seongjun Yun, Seoyoon Kim, Junhyun Lee, Jaewoo Kang, and Hyunwoo J. Kim. Neo-GNNs: Neighborhood Overlap-aware Graph Neural Networks for Link Prediction, 2022. URL https://arxiv.org/abs/2206.04216.

[31] Xiyuan Wang, Haotong Yang, and Muhan Zhang. Neural Common Neighbor with Completion for Link Prediction, 2023. URL https://arxiv.org/abs/2302.00890.

[32] Benjamin Paul Chamberlain, Sergey Shirobokov, Emanuele Rossi, Fabrizio Frasca, Thomas Markovich, Nils Hammerla, Michael M. Bronstein, and Max Hansmire. Graph Neural Networks for Link Prediction with Subgraph Sketching, 2022. URL https://arxiv.org/abs/2209.15486.

[33] Xiaohui Zhang, Yanbo Wang, Xiyuan Wang, and Muhan Zhang. Efficient Neural Common Neighbor for Temporal Graph Link Prediction, 2024. URL https://arxiv.org/abs/2406.07926.

[34] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs, 2020. URL https://arxiv.org/abs/2006.11468.

[35] Derek Lim, Felix Hohne, Xiuyu Li, Sijia Linda Huang, Vaishnavi Gupta, Omkar Bhalerao, and Ser-Nam Lim. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods, 2021. URL https://arxiv.org/abs/2110.14446.

[36] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A critical look at the evaluation of GNNs under heterophily: Are we really making progress?, 2023. URL https://arxiv.org/abs/2302.11640.

[37] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open Graph Benchmark: Datasets for Machine Learning on Graphs, 2020. URL https://arxiv.org/abs/2005.00687.

[38] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine, and Reihaneh Rabbany. Towards Better Evaluation for Dynamic Link Prediction, 2022. URL https://arxiv.org/abs/2207.10128.

[39] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. Temporal Graph Benchmark for Machine Learning on Temporal Graphs, 2023. URL https://arxiv.org/abs/2307.01026.

[40] Julia Gastinger, Shenyang Huang, Mikhail Galkin, Erfan Loghmani, Ali Parviz, Farimah Poursafaei, Jacob Danovitch, Emanuele Rossi, Ioannis Koutis, Heiner Stuckenschmidt, Reihaneh Rabbany, and Guillaume Rabusseau. TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs, 2024. URL https://arxiv.org/abs/2406.09639.